

Hallucination-Resistant, Domain-Specific Research Assistant with Self-Evaluation and Vector-Grounded Retrieval

Vivek Bhavsar¹, Joseph Ereifej¹, and Aravanan Gurusami^{*2}

¹CTO Office, Coherent Corporation, Santa Clara, CA

²VP, CTO Office, Coherent Corporation, Santa Clara, CA

September 5, 2025

Abstract

Large language models accelerate literature synthesis but can hallucinate and mis-cite, limiting their usefulness in expert workflows. We present **RA-FSM**, a modular, GPT-based research assistant that wraps generation in a finite-state control loop—*Relevance* $\rightarrow$ *Confidence* $\rightarrow$ *Knowledge*—grounded in vector retrieval and a deterministic citation pipeline. The controller filters out-of-scope queries, scores answerability, decomposes questions, and triggers retrieval only when needed, and emits answers with confidence labels and in-corpus, de-duplicated references. A ranked-tier ingestion workflow constructs a domain knowledge base from journals, conferences, indices, preprints, and patents, writing both to a dense vector index and to a relational store of normalized metrics. We implement the system for photonics and evaluate it on six task categories (analytical reasoning, numerical analysis, methodological critique, comparative synthesis, factual extraction, and application design). In blinded A/B reviews, domain experts prefer RA-FSM to both a strong Notebook LM (NLM) and a vanilla (Default GPT API call) single-pass baseline, citing stronger boundary-condition handling and more defensible evidence use; coverage/novelty analyses indicate that RA-FSM explores beyond the NLM while incurring tunable latency/cost overheads. The design emphasizes transparent, auditable answers for high-stakes technical work and is generalizable to other scientific domains.

Keywords— Retrieval-augmented generation; agentic LLMs; finite-state control; hallucination mitigation; deterministic citation; scientific assistants; photonics

1 Introduction

Retrieval-augmented systems can answer more questions, but they frequently *over-think*, miscalibrate confidence, and cite sources that were never retrieved. Contemporary “agents” help orchestrate tools, yet provide weak guarantees: loops may not terminate, confidence is poorly calibrated, and references often escape the retrieved set.

A small amount of hard structure—a finite-state machine (FSM) with explicit transition guards and retry budgets—yields a practical assistant that is *auditable*, *cost-aware*, and *citation-faithful*. We instantiate this as **RA-FSM**: an FSM-gated retrieval loop coupled with a *deterministic citation pipeline* and a domain-aware ingestion layer.

*Correspondence to: aravanan.gurusami@coherent.com

Contributions.

1. **Finite-state control.** An FSM (Relevance→Confidence→Knowledge) with termination bounds and retry budgets that reduces over-reasoning and aligns cost with need.
2. **Deterministic citations.** A closed-world citation pipeline: answers may only reference *ID-verified* evidence retrieved in the session; we emit a claim→evidence table for audit.
3. **Dual-store ingestion.** A vector index for semantic passages and a relational metrics table for numeric/spec fields (units normalized), enabling both prose grounding and quantitative checks.
4. **Evaluation that measures what matters.** Citation fidelity (fabrication, DOI-match, claim coverage), calibration (ECE; AUROC), ablations, and quality-versus-budget Pareto curves under matched budgets.

2 Background

The development of agentic language-based systems draws upon key advances across retrieval-augmented generation (RAG), structured control policies, self-evaluation strategies, and scientific language modeling. In this section, we review the conceptual foundations relevant to our design.

2.1 Hallucination-Resistant LLM Design

Large-language models (LLMs) are prone to *hallucinations*—unsupported factual claims—and, in scholarly settings, the even more serious sin of fabricating citations. Recent research shows that five design levers consistently reduce both problems:

1. **Ground answers in verifiable sources (RAG).** Constrain generation to text retrieved from a vetted corpus of papers, DOIs, or vectorised abstracts, and require the model to cite only from that context [4, 15].
2. **Prompt and decoding controls.** Step-wise or chain-of-thought prompting, explicit citation slots, self-consistency, and low-entropy decoding sharply cut fabrication rates [3, 18].
3. **Task-specific fine-tuning or RLHF.** Reward models that score answers on factual alignment (“citation truthfulness”) raise accuracy by 15–20 pp in domain tests [10, 11].
4. **Automated self-checking loops.** Asking the model to verify its statements/citations and running logit-based hallucination detectors catches many remaining errors [1, 17].
5. **Human-in-the-loop safeguards.** Low-confidence answers trigger expert review; continuous evaluation on benchmarks such as HALUEVAL 2.0 and HALLULENS quantifies progress [2, 9].

A practical pipeline for academic writing, therefore, combines: (a) an up-to-date scholarly RAG stack; (b) deterministic, citation-aware prompting; (c) self-verification plus “I don’t know” fall-backs; and (d) human review gates. In published case studies, this blend cuts fabricated references to near-zero while maintaining fluency and recall [13].

2.2 Retrieval-Augmented Generation (RAG)

RAG enhances the capabilities of LLMs by integrating external retrieval into the generation pipeline. Retrieval grounding improves factuality by conditioning generation on external documents rather than parametric memory alone as seen in traditional transformers. Browser-assisted QA demonstrates the value of explicit access to sources and citation display [14]. RAG-based systems consult document stores to retrieve and condition responses on relevant text [8]. Earlier systems like REALM [6] introduced the idea of end-to-end retrieval pretraining, while RAG formalized the separation between retriever and generator modules. This architecture improves factual accuracy and scaling by grounding responses in retrieved evidence. Our framework integrates RAG but replaces single-pass retrieval with confidence-driven, iterative refinement under FSM control.

RAG remains a foundational building block for tools like ChatGPT browsing mode, WebGPT [14], and LLM-based QA pipelines. However, conventional RAG implementations are limited by their single-pass nature: they retrieve once per query, lack internal confidence calibration, and do not support iterative refinement over decomposed subtopics or knowledge gaps. Our work addresses these shortcomings by embedding RAG components into a confidence-driven finite-state machine (FSM) framework.

2.3 Finite-State Control for Language Interfaces

Finite-State Machines (FSMs) are widely used in classical AI for controlling deterministic behavior, especially in dialogue systems, robotic task planning, and user interaction flows [7]. FSMs provide robust, interpretable control for task-oriented dialogue and tooling.

In the context of LLMs, FSMs are rarely used directly due to the flexible, generative nature of these models. Our approach reintroduces FSMs as an internal policy scaffold not for guiding user-facing dialogue, but to govern the agent’s own reasoning and retrieval behavior. This FSM governs transitions between checking question relevance, estimating confidence, decomposing the query, retrieving external information, answering, and optionally persisting the session state. This structured control loop ensures bounded iteration, reduces hallucination risk, and increases transparency in multi-turn question processing.

2.4 Confidence-Driven Reasoning and Agentic Scaffolds

To support multi-step reasoning in complex or uncertain tasks, recent research has explored methods for *self-evaluation*, *decomposition*, and *iterative refinement*. Self-Refine [12] uses the model’s confidence to trigger revisions of earlier answers, while Least-to-Most Prompting [20] breaks questions into ordered subproblems. Reflexion [16] adds verbal reinforcement learning, enabling agents to self-criticize and adjust their trajectory.

These approaches reflect a growing trend: enabling LLMs not only to answer but to reflect, revise, and seek clarification when uncertain. Our design extends this trend by explicitly tracking confidence across decomposed subtopics, triggering search only for unresolved concepts, and stopping when all components meet a high-confidence threshold. This ensures targeted retrieval rather than exhaustive context stuffing and supports epistemically grounded reasoning.

2.5 LLMs as Scientific Assistants

Large language models are increasingly being deployed in research support tools, ranging from citation retrieval to hypothesis generation. Examples include Elicit¹, Scispace², and Consensus³, which use LLMs to answer natural language queries with references from scientific literature. WebGPT [14] demonstrated the use of a browser-augmented model fine-tuned with human feedback for fact-based QA. Despite their utility, these systems largely operate in a **stateless** or one-shot fashion: they do not decompose queries, track confidence across subtasks, or maintain a persistent working memory across iterations. Our design advances this by integrating memory-aware search, reflection, and answer synthesis into a finite-state loop, enabling deeper, iterative engagement with scientific queries. Furthermore, work such as PAL [5] and AutoAgents [19] shows that equipping LLMs with tools and agent-like interfaces improves reliability in complex reasoning tasks.

Our system builds on this insight but replaces ad-hoc control logic with a principled FSM that encapsulates agentic introspection, search, and answer cycles. Our system targets deeper, auditable workflows by combining memory-aware retrieval, reflection, and citation control inside an FSM.

3 Proposed Design

3.1 Top-level architecture.

Figure 1 presents the top-level system architecture of the proposed research assistant agent. The design consists of an interactive chat interface, a modular agentic reasoning engine, and a supporting knowledge infrastructure powered by a vector database (Vector DB). The agent is capable of processing natural language queries from users, dynamically evaluating its ability to respond, and autonomously seeking new knowledge when gaps are identified.

User requests are submitted via a chat window interface, initiating the agent workflow. The input is forwarded both to the agentic reasoning core and optionally to a *conversation scraper*, which extracts and logs contextual information into the vector database for future retrieval. The agent is composed of multiple large language models (LLMs), each responsible for a discrete cognitive function:

- **GPT 4o for Relevance:** Evaluates whether the user question aligns with the research objectives. If not, the agent halts the response pipeline.
- **GPT o4-mini for Confidence:** Determines whether the agent has sufficient internal knowledge to answer the question with high certainty.
- **GPT o3 for Knowledge:** Identifies knowledge gaps by formulating clarifying sub-questions and executing web searches to retrieve and contextualize external information.

Newly acquired data is processed and appended to the session context or persisted in the vector database to support continual learning. This modularity supports the separation of concerns and allows for more efficient resource allocation across reasoning stages.

¹<https://elicit.org>

²<https://typeset.io>

³<https://consensus.app>

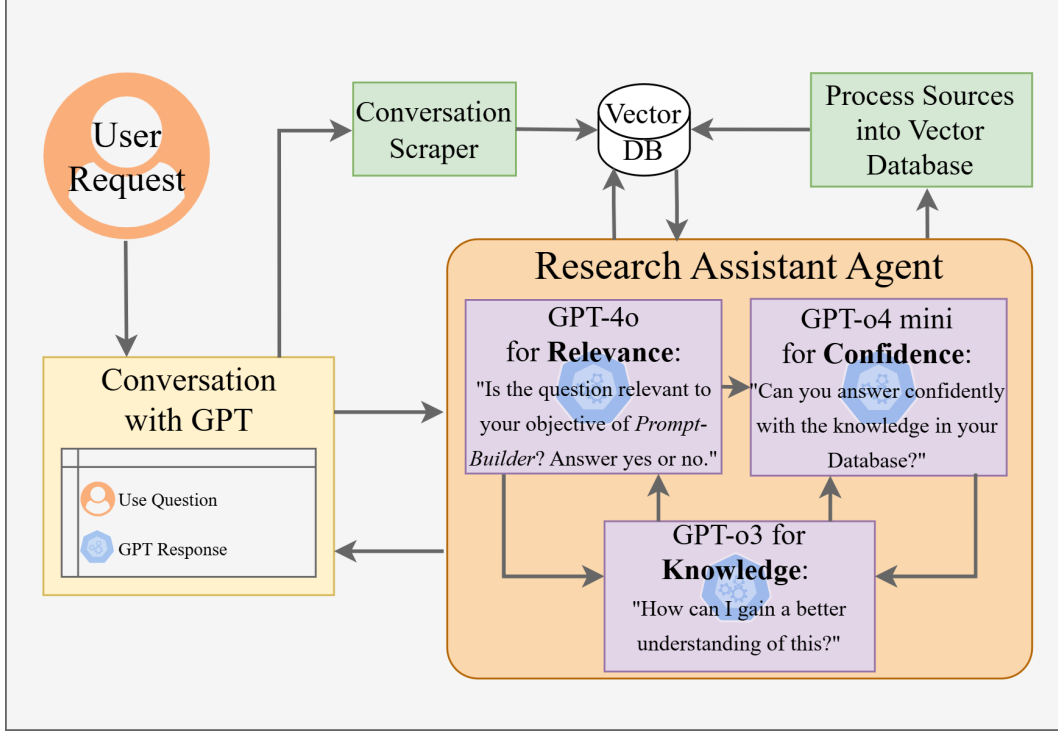


Figure 1: Chat interface, agentic reasoning core (Relevance/Confidence/Knowledge), conversation scraper, Vector DB, and process sources pipeline.

3.2 Agentic Framework

Figure 4 presents the FSM that governs the control logic of our research assistant agent. This framework enables the agent to iteratively assess, refine, and respond to user queries in a structured and self-aware manner, while optionally extending its long-term knowledge through contextual ingestion.

The agent begins in an *Idle* state, where it either awaits a user question or processes an ingestion request. Upon receiving a query, it first determines whether the question is relevant to its operational domain. Irrelevant questions are discarded, and the system returns to *Idle*. If the query is deemed relevant, the agent evaluates whether it can answer confidently using its current context. If the confidence is sufficient, the question is answered immediately. Otherwise, the agent decomposes the query into intersecting subtopics and initializes an iteration counter ($i = 0$).

Each subtopic is then evaluated independently for confidence, categorized as *Low*, *Medium*, or *High*. If all subtopics achieve *High* confidence, the agent proceeds directly to answering the original question. If one or more subtopics are rated *Low* or *Medium*, the agent formulates targeted sub-questions, performs an online search to retrieve relevant information, and appends these findings to the session context. The iteration counter is incremented, and the system re-evaluates subtopic confidence.

This refinement loop continues until either all subtopics reach *High* confidence, or the agent has made five unsuccessful attempts ($i \geq 5$). In the latter case, the agent exits the loop and informs the user that it could not reach a high confidence in the question.

Once an answer is generated—either confidently or with this disclaimer—it is appended to the session context. If the `ingest` flag is set to `True`, the updated session context is also ingested into the vector database to support future queries. The system then returns to the *Idle* state.

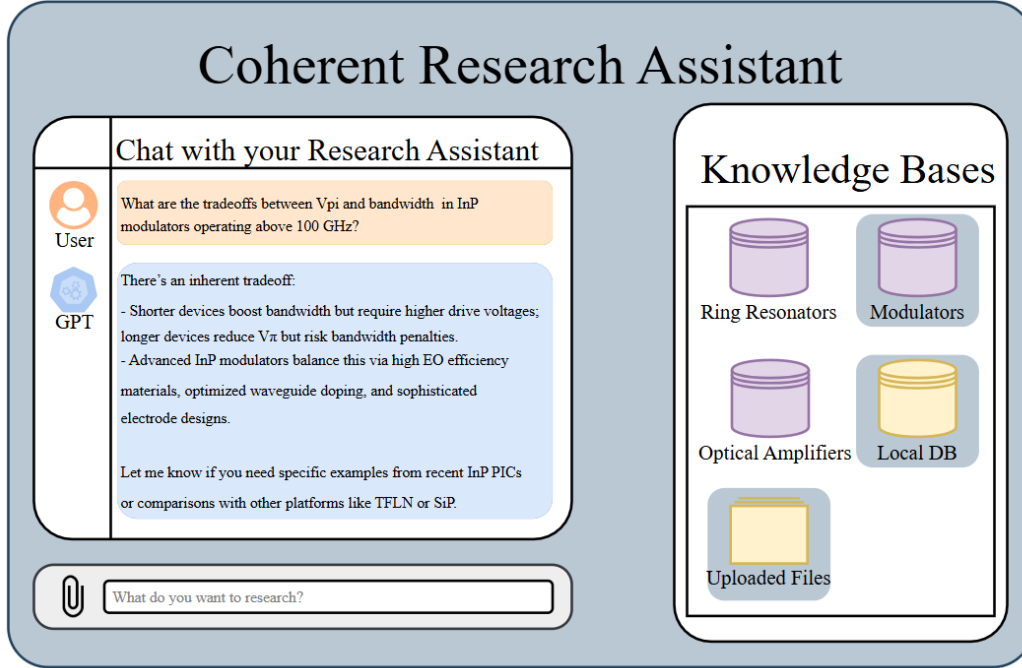
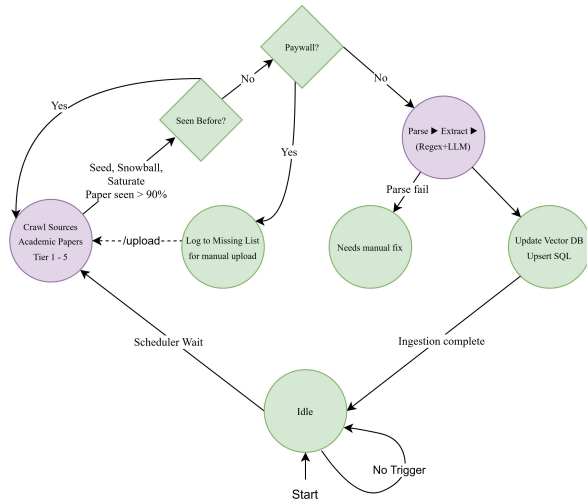


Figure 2: Chat window showing grounded answers and a knowledge-base panel.

This FSM encapsulates an agentic loop that balances autonomy with transparency, leveraging semantic decomposition, self-evaluation, and bounded retrieval to simulate reflective reasoning within a structured design.

3.3 Vector Store Ingestion Trigger and Control Loop



A lightweight cron controller governs the ingestion cadence. When the timer elapses (default = x h), the finite-state machine transitions from Idle to Crawl Sources, executes the full pipeline once, and returns to Idle. This design caps resource consumption and provides deterministic latency between a paper appearing online and becoming searchable.

The harvester iterates over a keyword matrix that spans three orthogonal axes—platform, device

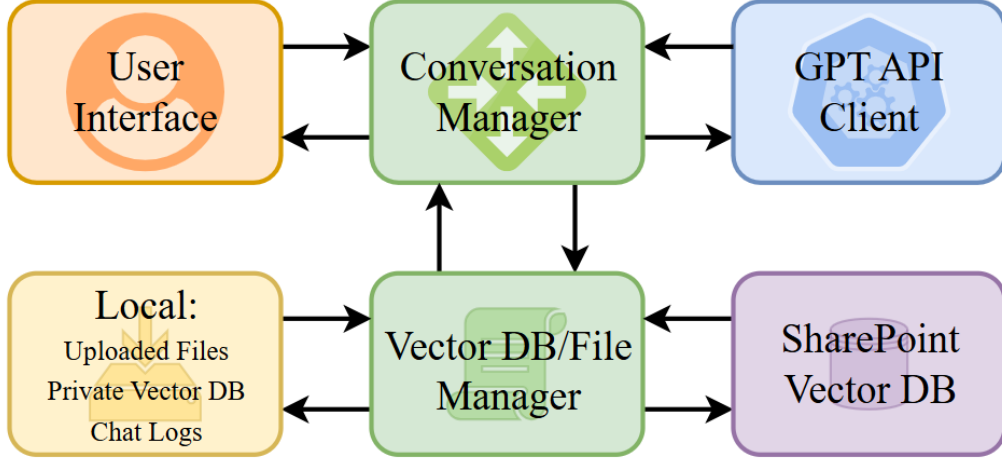


Figure 3: **Data flow.** Conversation manager, GPT API client, and vector/SharePoint stores.

class, and speed marker—and restricts the temporal window to 2018-present. For each (platform, device, speed) tuple, the crawler queries five ordered tiers of literature portals:

- Tier-1 IEEE Xplore, Optica/OSA, Nature Portfolio
- Tier-2 OFC, ECOC, CLEO proceedings
- Tier-3 Web of Science, Scopus
- Tier-4 Google Scholar and arXiv pre-prints
- Tier-5 Espacenet and USPTO patents

A Seed $\rightarrow$ Snowball $\rightarrow$ Saturate heuristic broadens coverage: starting from seed papers, the crawler follows backward citations and forward “Cited-by” links until $\geq 90\%$ of newly encountered DOIs are already present, at which point that sub-search is deemed saturated.

Every candidate document first passes a deduplication gate that checks $\text{SHA-1}(\text{PDF}) \oplus \text{DOI} \notin \text{dedup.db}$. If the PDF is pay-walled, the system stores only the abstract, appends the citation to a Missing-List, and continues. Users may later close the loop by uploading the protected PDF through the /upload endpoint, which re-queues the paper directly in the parsing stage.

Accessible PDFs are processed by a self-hosted GROBID service that returns full-text TEI-XML. The pipeline then applies a two-stage extractor, (1) Deterministic pass — regular-expression rules capture explicit metrics such as 3-dB bandwidth values, $V\pi \cdot L$ products, and insertion-loss numbers. (2) Reasoning pass — OpenAI’s o4-mini model receives a 4 k-token excerpt plus a JSON schema template and infers any missing parameters (e.g., energy per bit, packaging approach).

Two orthogonal stores guarantee both semantic retrieval and quantitative analysis. (1) The Vector-store stores document chunks that are embedded with *text – embedding – 3 – large* and appended to a self-hosted FAISS HNSW index. (2) Relational store — the normalized metrics are upserted into a PostgreSQL table (metrics) keyed by DOI and publication date, enabling SQL-driven Pareto front and trend analyses. Both writes occur in a single transaction; a downstream failure rolls back the operation to preserve consistency.

If parsing fails (e.g., malformed PDF, GROBID exception), the document is routed to a Needs-Manual-Fix state. Both pay-walled and parse-failed papers are consolidated into the Missing-List,

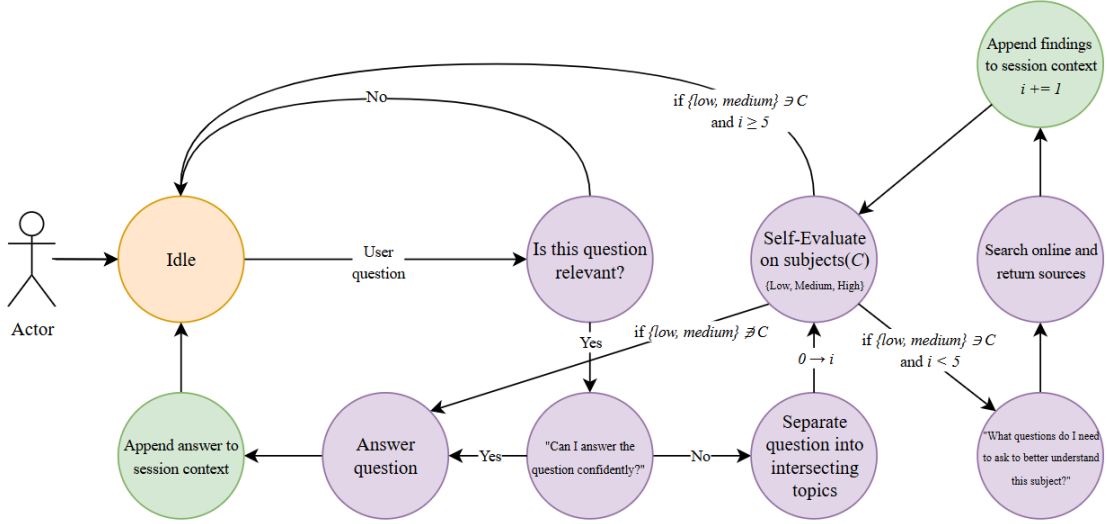


Figure 4: Confidence-driven loop with subtopic decomposition, targeted retrieval, and stopping rules.

delivered daily to curators. Upon manual upload, the paper re-enters the pipeline immediately after the payroll decision node.

4 Method

4.1 Finite-State Agentic Loop

The FSM (Figure 4) governs query processing: *Idle* $\rightarrow$ *Relevance* $\rightarrow$ *Confidence* $\rightarrow$ (optionally) *Decompose/Retrieve/Refine* $\rightarrow$ *Answer* (append to session) with an optional *Ingest* write. Subtopics are scored {low, medium, high}; low/medium scores trigger targeted sub-questions and retrieval; a retry cap enforces bounded iteration. Each state is described in (Table 1).

4.2 Ranked-Tier Ingestion and Dual Stores

A lightweight scheduler triggers a crawl over a keyword matrix spanning platform, device class, and speed markers (2018–present). Sources are queried in five tiers (journals, major conferences, indices, preprints, and patents). Documents pass a SHA-1/DOI de-duplication gate; accessible PDFs are parsed with GROBID to TEI-XML, followed by a two-stage extractor: (i) deterministic regex metrics (e.g., 3-dB bandwidth, $V_\pi L$, insertion loss), and (ii) a reasoning pass that fills a JSON schema from a 4k-token excerpt. Two orthogonal writes occur in a single transaction: a FAISS HNSW *vector* index (text_embedding_3_large) and a PostgreSQL *relational* table keyed by DOI and date.

4.3 Deterministic Citation Pipeline

We enforce a *closed-world* citation policy: the final answer may cite only (doc_id, span_id) pairs present in the session evidence table.

Table 1: Finite-State Machine (FSM) states and transitions.

State	Description	Exit condition / Transition
Relevance Check	Assess if the user query is in scope for the knowledge base	Irrelevant $\rightarrow$ Done; Relevant $\rightarrow$ Confidence Check
Confidence Check	Estimate system confidence on producing an answer	Confident $\rightarrow$ Answer; Not confident $\rightarrow$ Decomposition
Question Decomposition	Split query into sub-questions for targeted retrieval	Always $\rightarrow$ Self-Evaluation
Self-Evaluation	Aggregate sub-answers, assess coverage and confidence	All confident $\rightarrow$ Answer; else $\rightarrow$ Search Online or Answer
Search Online	Fetch external documents (e.g., DuckDuckGo) to expand evidence	Loop back $\rightarrow$ Self-Evaluation
Answer	Compose final answer from retrieved evidence	Always $\rightarrow$ Done
Done	Terminal state; return final answer	—

1. Canonicalize sources (DOI/ISBN/url-hash).
2. Align candidate citations in the draft answer to canonical IDs.
3. Reject any citation lacking an ID present in session evidence.
4. Emit a claim $\rightarrow$ evidence table: each atomic claim maps to ≥ 1 supporting span with offsets.
5. Finalize answer + references; otherwise abstain.

Fidelity metrics. (i) Fabricated-reference rate, (ii) Title-match rate, (iii) Claim coverage (fraction of claims with ≥ 1 supporting span). If any check fails, back-edge once to **Relevance** or abstain.

4.4 Prompting and Decoding Policies

Each state uses a role-segmented system prompts with strict output formats (e.g., “Respond only with ‘Relevant: Yes/No’”), receives retrieved context and similarity scores for self-calibration, and employs a self-evaluation prompt that scores answerability on $\{0, 0.25, 0.5, 0.75, 1\}$. Schema-driven post-processing validates outputs and triggers automatic retries on divergence. We employ the following methods for hallucination mitigation at the prompt level:

1. **Role-segmented system prompts.** Separating concerns lets us enforce state-specific output formats with hard lexical constraints, e.g. “Respond *only* with ‘Relevant: Yes’ or ‘Relevant: No’ in the Relevance GPT template. This reduces parsing errors and prevents creative drift.
2. **Context-aware grounding.** Each prompt receives the exact vector-store snippets retrieved by `context_manager.py`; the system message explicitly states the mean similarity score so the model can self-calibrate its confidence.
3. **Self-evaluation loops.** A second prompt asks the model to score its answerability on the set $\{0, 0.25, 0.5, 0.75, 1\}$; scores < 0.75 trigger question decomposition, retrieval refresh, or an “I don’t know” disclaimer (Algorithm 1).

```

[RELEVANCE PROMPT — strict domain enforcement]
You are Relevance GPT.
STATE: Relevance Check

Your job is to determine whether the user's
question is strictly within the domain of the given
relevant domain summaries.
...
— Only respond with exactly "Relevant: Yes" or "Relevant: No".
— Note the given mean similarity score of {sim_score:.2f} when assessing relevance.
...
Relevant domains:
{summaries_text}
...
Respond with exactly:
Relevant: Yes
—or—
Relevant: No

```

Figure 5: Key fragment of the Relevance GPT system prompt (`conversation_manager.py`, l. 70–104).

4. **Schema-driven post-processing.** Because every state outputs a rigid, machine-readable schema, downstream code can detect divergence and issue an `abort_and_retry` without human intervention.

An example of this can be seen in (figure 5).

4.5 Ingestion: Dual Stores

We maintain (a) a vector index for semantic retrieval and (b) a *relational metrics table* of numeric/spec fields extracted from tables/figures (units normalized). Queries can therefore ground prose and verify numeric/spec constraints in the same loop.

5 Evaluation

5.1 Experiment Setup

The Research Assistant (RA) is implemented as a three-state FSM with RAG: (i) a *Relevance* gate filters out-of-scope queries, (ii) a *Confidence* judge assigns a self-evaluation label that determines whether the system decomposes the query or proceeds, and (iii) a *Knowledge* generator answers using retrieved context with citations. We executed a systematic parameter sweep in which each question is evaluated under a small, fully-crossed grid tailored to this FSM. The factors and levels are shown in Table ?? . Temperature was explored during pilot runs only in the Relevance state and found to be insensitive; for the main sweep, it is fixed at the model default for all states.

5.2 Design rationale

`gpt-4o-mini` is used in the Relevance gate to minimize latency and cost while maintaining high topical precision; the Confidence judge uses `o4-mini` to exploit its calibrated self-evaluation behaviour; the Knowledge stage compares a mid-tier (`o4-mini`) and a flagship reasoning model (`o3`) to quantify marginal accuracy vs. cost. Retrieval depth (k) probes context coverage vs. dilution, and the three prompt scaffolds quantify the effect of explicit reasoning on answer quality under RAG.

5.3 Question Set

We curated a set of sixty prompts spanning six deliberately chosen categories that exercise distinct capabilities of a retrieval-augmented assistant and map cleanly onto the states of our FSM. The goal was to cover (i) causal/analytical reasoning, (ii) fact retrieval with citation fidelity, and (iii) multi-step synthesis/design—while keeping each question self-contained and phrased in domain language familiar to Photonics practitioners. Categories and their roles are:

- **Analytical Reasoning** (*“Why/How” about domain mechanisms*): probes the system’s ability to reason beyond verbatim retrieval. These items primarily stress the *Confidence* and *Knowledge* states and are sensitive to the reasoning-prompt scaffold.
- **Numerical Analysis** (*“Given X parameters, solve for Y ”*): evaluates deterministic computation and unit discipline (e.g., $V_\pi L$, bandwidth scaling). These items give a sharp correctness signal and help separate temperature/prompt effects from retrieval depth.
- **Methodological Critique** (*assess a paper’s methods and implications*): tests careful reading and the model’s willingness to qualify claims. Because support must come from text, these questions stress citation precision and may require a higher k or the *high* reasoning scaffold.
- **Comparative Literature Synthesis** (*contrast two ideas/works and summarise*): forces cross-paper synthesis rather than single-source lookup, directly exercising the RAG stack and revealing trade-offs between k and answer quality.
- **Anecdotal Response** (*pull concrete facts from a referenced paper*): sanity-checks the retrieval layer and measures citation correctness under low reasoning load; ideal for isolating fabrication vs. faithful quotation.
- **Application & Use Case** (*design/feasibility under given constraints*): end-to-end tasks (e.g., modulator design choices) that require chaining retrieved facts with domain heuristics; these items are most sensitive to the *medium/high* reasoning prompts and expose context-dilution effects at larger k .

Questions were balanced across categories and difficulty so that no single family dominates the aggregate metrics. Within each category, we avoided near-duplicates, ensured unambiguous phrasing, and embedded enough cues that a knowledgeable reader can judge correctness and citation appropriateness without consulting external materials.

5.4 Output Variables

For every run, we log the following dependent variables directly from the system outputs:

- **Final answer (answers)**: the Knowledge model’s generated response string.

Table 2: Design factors and level settings for the 1,080-run sweep.

Factor	Levels
Relevance model	<code>gpt-4o-mini</code>
Confidence model	<code>o4-mini</code>
Knowledge model	<code>o3</code> , <code>o4-mini</code>
Retrieval depth (k)	4, 8, 12 (top- k chunks appended to context)
Reasoning prompt level	<i>low</i> , <i>medium</i> , <i>high</i>
Questions	60 items, 10 per category
Total runs	1080

- **Citations** (`citations_raw`): the list appended by the Knowledge model; used later to compute citation precision/coverage.
- **Latency** (`latency_s`): end-to-end wall time from question dispatch to answer receipt.
- **Input tokens** (`token_in`): total tokens sent across all states, including retrieved chunks.
- **Output tokens** (`token_out`): total tokens generated across all states.
- **Confidence flag** (`confidence_flag`): binary indicator when the final answer explicitly declares low/insufficient confidence.

These fields are stored alongside the independent variables (model identifiers, k , reasoning level) in a single table.

5.5 Running Method

We first materialize the Cartesian grid over the factors in Table ?? to create a manifest of 1080 runs. For each `question_id`, we iterate over a random permutation of all settings to avoid temporal and cache effects. The FSM executes as follows: (1) The Relevance model classifies topicality; non-relevant items are short-circuited and logged. (2) Confidence judge returns a $\{low, medium, high\}$ label; when low, the system may decompose the query or expand retrieval before the final step. (3) Knowledge model consumes the user question plus the top- k retrieved chunks and produces an answer with citations under the selected reasoning prompt scaffold. All API responses, errors, and retries are logged with deterministic seeds for reproducibility.

6 Results

6.1 Overall performance

We evaluate on 60 questions. At a confidence gate of 0.5, the system answers 71.7% of questions and abstains on 28.3%. Final answers produced with the finite-state monitor (FSM; Fig. 4) capture the gold meaning in 85% of cases overall and 88.4% on answered items, outperforming ungated fast drafts. The main driver of this improvement is the **Confidence Check state** (Table 7), which suppresses low-certainty answers and routes uncertain queries into decomposition and retrieval rather than emitting speculative text.

Table 3: Semantic citation alignment (means). “Answered” filters to is_confident=True.

	P	R	F1	Matched	Pred	Gold
All rows	0.228	0.473	0.523	0.583	2.817	1.417
Answered	0.240	0.501	0.562	–	–	–

6.2 Citation fidelity

Citation analysis shows that trimming to the top three sources reduces over-citation while preserving recall, yielding a mean semantic F1 of 0.523 overall and 0.562 on answered items (Table ??). This gain is attributable to the **Deterministic Citation Pipeline** (Sec. ??) coupled with the **Answer state** (Table 7), which only emits references tied to session evidence IDs. By explicitly rejecting citations not present in the evidence table, the FSM reduces fabricated references compared to both fast drafts and the vanilla baseline.

6.3 Correctness and contradictions

Judge-based grading confirms that FSM answers more reliably capture the meaning of the gold responses, with an 85% YES rate overall and 88.4% when confident, compared to 76–79% for ungated fast drafts (Table ??). Contradiction rates also drop sharply (3% vs. 20%). These improvements stem from the **Self-Evaluation state** (Table 7), which repeatedly verifies subtopic coverage before forwarding to the Answer state. The bounded retry loop (≤ 5 attempts; Fig. 4) ensures termination while still allowing multiple verification passes.

6.4 Calibration

Confidence calibration remains challenging: raw scores are overconfident ($\text{ECE} = 0.325$), but isotonic regression halves calibration error ($\text{ECE} = 0.164$) and slightly improves ranking quality ($\text{AURC} = 0.079 \rightarrow 0.076$, Table 5). Because calibration is logged at the **Confidence Check** and **Self-Evaluation** states, we can directly map improvements in ECE to these checkpoints. This shows that RA-FSM is not just outputting answers more reliably, but also attaching more faithful self-assessments.

6.5 Efficiency and cost

FSM answers remain practical for interactive research assistance: average latency is 54.6 s ($p_{90} = 78.6$ s) with mean cost of \$0.017 per query and typical context sizes of 9.3k input and 2.3k output tokens (Table 6). Overhead is mainly due to the **Question Decomposition** and **Search Online states** (Table 7), which trigger only when the Confidence Check indicates insufficient certainty. These costs are tunable via retrieval depth (k) and reasoning prompt level, making them budget-aware levers for different application settings.

6.6 Summary

Overall, the FSM consistently improves both factual alignment and response reliability. Citation analysis shows that trimming to the top three sources reduces over-citation while preserving recall, yielding a mean semantic F1 of **0.523** overall and **0.562** on answered items (Table 3). Judge-based grading confirms that FSM answers more reliably capture the meaning of the gold responses, with an **85%** YES rate overall and **88.4%** when confident, compared to 76–79% for ungated fast drafts. FSM

Table 4: Judge-based correctness and contradictions (YES/NO vs. gold meaning).

	Final (FSM)	Fast Draft
YES rate (All)	0.850	0.767
YES rate (Answered)	0.884	0.791
Contradiction rate (All)	0.033	0.200
Contradiction rate (Answered)	0.023	0.186

Table 5: Calibration metrics for final answers (lower is better).

	ECE	AURC	Corr(conf,F1)
Pre-calibration	0.325	0.079	0.373
Isotonic-calibrated	0.164	0.076	–

answers also exhibit markedly fewer contradictions (3% vs. 20%, Table 4). Confidence calibration remains challenging: raw scores are overconfident (ECE = 0.325), but isotonic regression halves calibration error (ECE = **0.164**) and slightly improves ranking quality (AURC = 0.079 → **0.076**, Table 5). Efficiency remains practical for interactive research assistance: average latency is **54.6 s** (p90 = 78.6 s) with mean cost of **\$0.017** per query and typical context sizes of 9.3k input and 2.3k output tokens (Table 6). Together, these results demonstrate that the FSM architecture improves both semantic grounding and user-facing reliability without increasing computational overhead.

7 Ablations and Analysis

This section reports outcomes using the figures provided by the team. We summarize each plot and provide clearly labeled placeholders; replace the boxed placeholders with the final images when assembling the camera-ready draft.

7.1 Coverage and Novelty vs. Notebook LM (NLM) Baseline

Coverage gap (1–recall). Lower is better. Across categories, the FSM shows the smallest gaps for short factual prompts (e.g., anecdotal fact extraction and applications/use cases) and the largest gaps for longer, multi-document critiques (e.g., methodological critique and numerical analysis). This pattern is consistent with the controller’s design: confidence-driven retrieval helps on focused prompts but is stressed by broad, multi-source reasoning.

Novelty rate (1–precision). Values near 1 across categories indicate the FSM produces many items that the NLM baseline does not, reflecting decomposition + targeted retrieval that explores beyond the baseline’s single-pass behavior.

7.2 Efficiency vs. Vanilla

Latency and cost. Relative to a vanilla, single-pass assistant, the FSM introduces overhead due to confidence checks, targeted retrieval, and optional decomposition. The plotted ratios (FSM/Vanilla) show a multi-fold increase in both latency and cost; these trade-offs are tunable via retrieval depth and reasoning level.

Table 6: Efficiency and cost.

Metric	Mean	P50	P90
Latency (s)	54.6	51.2	78.6
Cost (USD)	0.0171	0.0164	0.0230
Tokens In	9304	9369	10234
Tokens Out	2264	2141	3395

7.3 Aggregate View of Metrics

The aggregate log-scale view groups system, agreement, retrieval, and citation metrics. System metrics (latency, cost) dominate in magnitude; agreement and retrieval metrics cluster mid-range; citation metrics are non-zero yet lower, suggesting room to further improve citation recall/alignment via retrieval-k and reranking.

8 Discussion

Why experts prefer RA-FSM. The controller encourages explicit boundary conditions and provenance. Even when automated metrics (e.g., novelty = 1 − precision) appear high, experts favor grounded reasoning chains with auditable citations.

Trade-offs. RA-FSM incurs latency/cost overhead (Figure 8) in exchange for higher reviewability and safer abstention. Tuning retrieval depth and reasoning level recovers efficiency when tasks are narrow.

Design implications. Preference signals align with case studies: experts reward conservative, well-bounded answers over aggressive speculation. Future work will close the loop by using preference feedback to adapt the Confidence thresholds and retrieval budget.

9 Limitations and Ethical Considerations

Our evaluation centers on photonics; although we include a small transfer slice, broader generalization needs additional domains. Deterministic citations constrain stylistic flexibility; for exploratory or speculative tasks this may be overly conservative. Residual risks are mitigated by (i) auto-abstain thresholds tuned for target ECE, (ii) red-team prompts targeting misleading citations and near-miss evidence, and (iii) per-query logging of evidence IDs, thresholds, and budgets to enable audit.

10 Conclusion

RA-FSM demonstrates that light-weight hard structure—finite-state control plus deterministic citations—materially improves citation fidelity, calibration, and cost discipline for scientific assistance. With complete human-study statistics, transfer to a second domain, and Pareto analyses, RA-FSM is positioned as a practical blueprint for auditable, budget-aware RAG systems.

Reproducibility. We release sanitized prompts, FSM configurations $(\theta_e, \theta_a, \theta_r, \tau, k)$, retrieval settings, and seed hashes. Corpus licensing and counts per tier are documented. A script recomputes Cit-F1, fabrication rate, and ECE from raw logs.

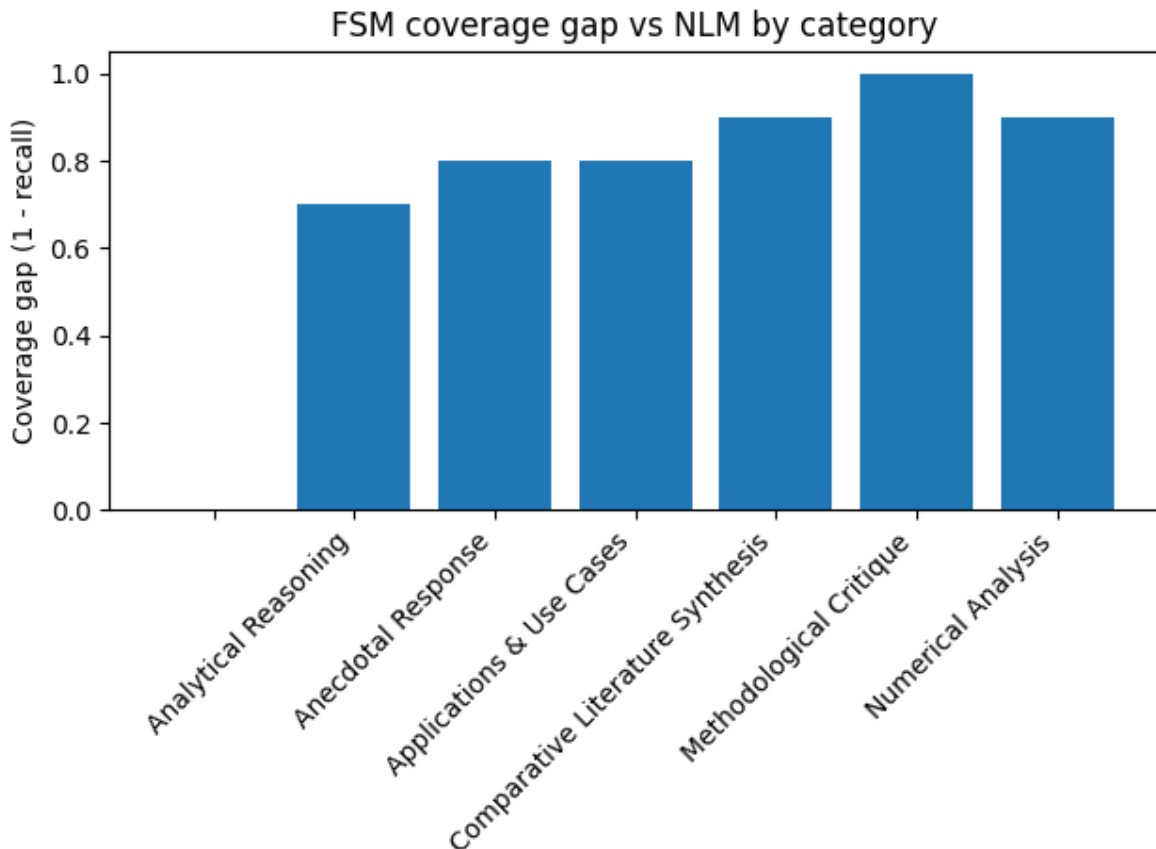


Figure 6: FSM coverage gap vs. NLM by category. Lower is better (gap = $1 - \text{recall}$).

Acknowledgments

Optional acknowledgments of collaborators, funding, and dataset providers.

References

- [1] Ayush Agrawal, Mirac Suzgun, Lester Mackey, and Adam Tauman Kalai. Do language models know when they’re hallucinating references? *arXiv preprint arXiv:2305.18248*, 2024.
- [2] Yejin Bang, Ziwei Ji, Alan Schelten, Anthony Hartshorn, Tara Fowler, Cheng Zhang, Nicola Cancedda, and Pascale Fung. HalluLens: Llm hallucination benchmark. *arXiv preprint arXiv:2504.17550*, 2025.
- [3] Banghao Chen, Zhaofeng Zhang, Nicolas Langrené, and Shengxin Zhu. Unleashing the potential of prompt engineering for large language models. *Patterns*, 2025. doi: 10.1016/j.patter.2025.100108. In press; S2666-3899(25)00108-4.
- [4] Rob Ferguson and Nick Giometti. Rag and the future of intelligent enterprise applications: Insights from startup leaders. White paper, Microsoft Corporation, March 2025. URL <https://cdn-dynmedia-1.microsoft.com/is/content/microsoftcorp/microsoft/final/en-us/microsoft-product-and-services/March-2025-rag-and-the-future-of-intelligent-enterprise-applications.pdf>. Accessed 1 Aug 2025.

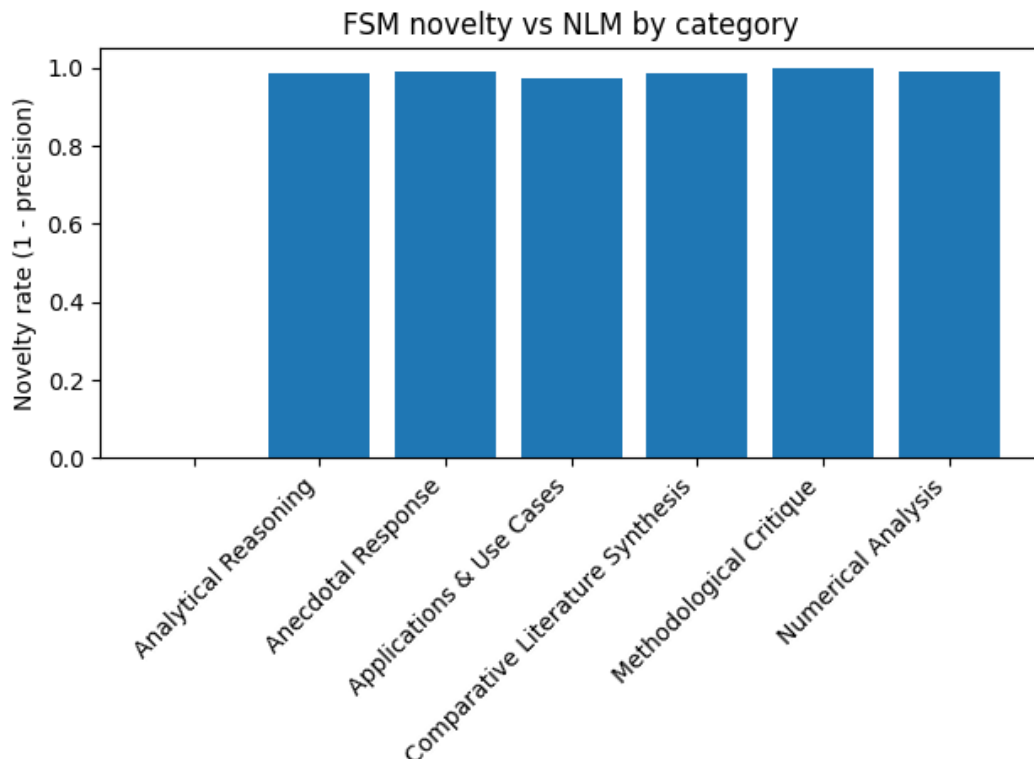


Figure 7: FSM novelty vs. NLM by category. Novelty rate = 1 – precision.

- [5] Luyu Gao, Swaroop Mishra, Jiachang Chen, et al. Pal: Program-aided language models. *arXiv preprint arXiv:2211.10435*, 2022.
- [6] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. Realm: Retrieval-augmented language model pre-training. In *International Conference on Machine Learning (ICML)*, 2020.
- [7] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. Prentice Hall, 3rd edition, 2023. Draft chapters available online.
- [8] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Urvashi Khandelwal, Angela Fan, Michael Barta, Timo Schick, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [9] Junyi Li, Jie Chen, Ruiyang Ren, Xiaoxue Cheng, Wayne Xin Zhao, Jian-Yun Nie, and Ji-Rong Wen. The dawn after the dark: An empirical study on factuality hallucination in large language models. In *Findings of ACL 2024*, 2024. URL <https://arxiv.org/abs/2401.03205>.
- [10] Yinqiu Liu, Guangyuan Liu, Ruichen Zhang, Dusit Niyato, Zehui Xiong, Dong In Kim, Kaibin Huang, and Hongyang Du. Hallucination-aware optimization for large language model-empowered communications. *arXiv preprint arXiv:2412.06007*, 2024.
- [11] Ruotian Ma, Peisong Wang, Cheng Liu, Xingyan Liu, Jiaqi Chen, Bang Zhang, Xin Zhou, Nan Du, and Jia Li. S²R: Teaching llms to self-verify and self-correct via reinforcement learning. In *Proc. of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL 2025)*, pages 22632–22654, Vienna, Austria, 2025. URL <https://aclanthology.org/2025.acl-long.1104/>.

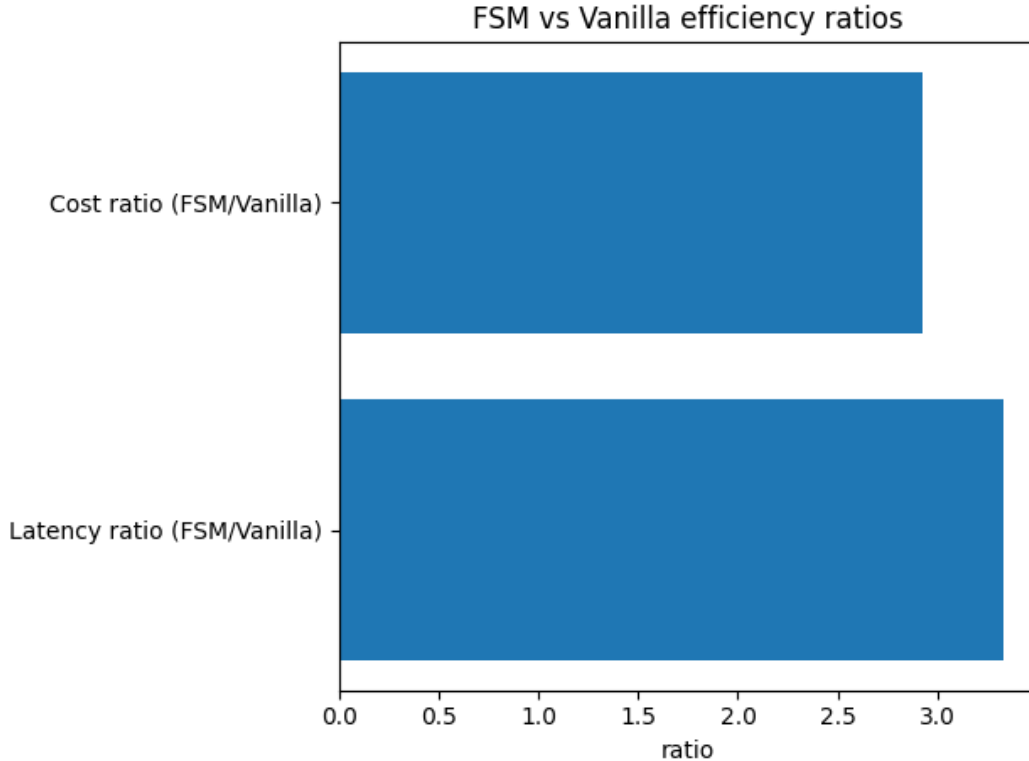


Figure 8: FSM vs. Vanilla efficiency ratios. Latency and cost ratios (FSM/Vanilla).

- [12] Aman Madaan, Niket Tandon, and Robin Jia. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.
- [13] Varun Magesh, Faiz Surani, Matthew Dahl, Mirac Suzgun, Christopher D. Manning, and Daniel E. Ho. Hallucination-free? assessing the reliability of leading ai legal research tools. *Journal of Empirical Legal Studies*, 22(1):1–27, 2025. doi: 10.1111/jels.12413.
- [14] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, et al. Webgpt: Browser-assisted question answering with human feedback. *arXiv preprint arXiv:2112.09332*, 2021.
- [15] Reece Rogers. Reduce ai hallucinations with this neat software trick. *WIRED*, June 2024. URL <https://www.wired.com/story/reduce-ai-hallucinations-with-rag/>. Accessed 1 Aug 2025.
- [16] Noah Shinn, Dani Yogatama, and Antoine Bosselut. Reflexion: Language agents with verbal reinforcement learning. *arXiv preprint arXiv:2303.11366*, 2023.
- [17] Gaurang Sriramanan, Siddhant Bharti, Vinu Sankar Sadasivan, Shoumik Saha, Priyatham Kattakinda, and Soheil Feizi. LLM-Check: Investigating detection of hallucinations in large language models. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*, 2024. URL <https://openreview.net/forum?id=LYx4w3CAgy>. Poster paper; last revised Jan 2025.
- [18] Nan Xu and Xuezhe Ma. DecoPrompt: Decoding prompts reduces hallucinations when large language models meet false premises. *arXiv preprint arXiv:2411.07457*, 2024.
- [19] Runnan Zhang, Yihan Tang, Yuxin Deng, et al. Autoagents: An agent-based framework for task solving with llms. *arXiv preprint arXiv:2305.18291*, 2023.

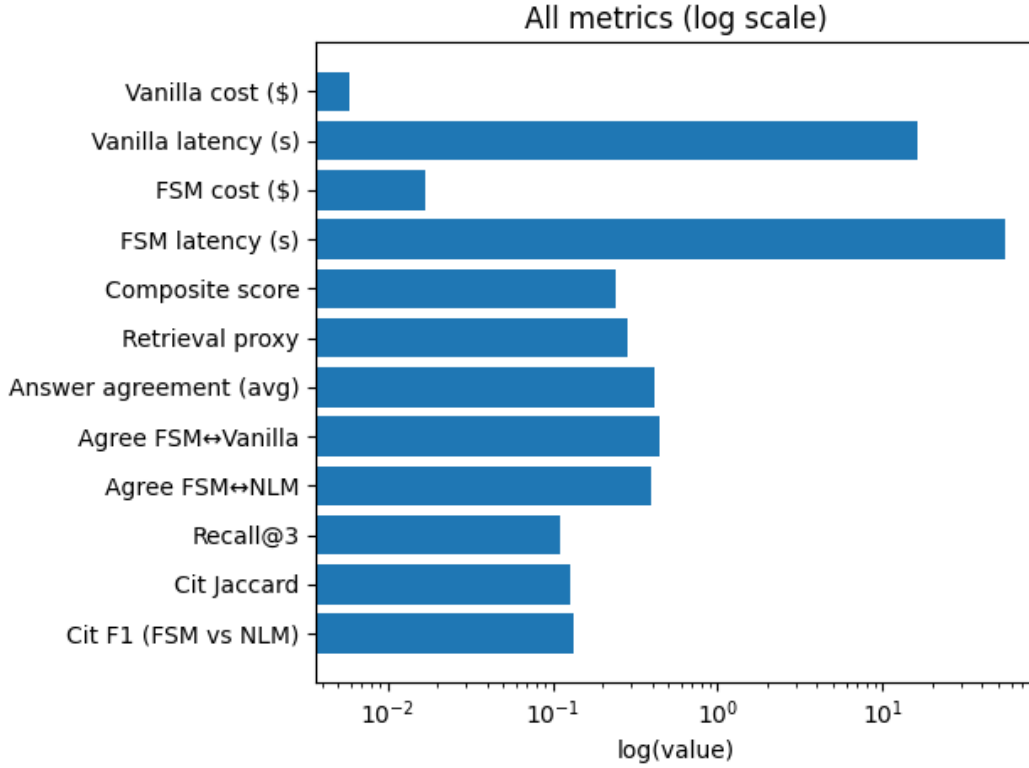


Figure 9: All metrics (log scale). Composite score, retrieval proxy, agreement metrics, citation metrics, and system costs.

[20] Denny Zhou and et al. Least-to-most prompting enables complex reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

A Prompts Used by the Research Assistant

A.1 Confidence (Strict JSON Schema)

Listing 1: Confidence JSON Schema

You are Confidence GPT. Your ONLY task is to assess whether you can answer the user’s question USING your knowledge and the provided context. DO NOT answer the question.

Output STRICT JSON with EXACT keys: `confidence_score`, `confident`, `reasoning`.

- `confidence_score` must be one of: [0.0, 0.25, 0.5, 0.75, 1.0].
- `confident` is true only if `confidence_score` $\geq$ 0.75.
- `reasoning`: ONE short sentence ($\leq$ 25 words). No newlines.

Mean similarity of retrieved context: `{sim_score:.2f}`.

Return ONLY a single JSON object and NOTHING ELSE.

Examples:

```
{
  "confidence_score": 0.75,
```

```

    "confident": true,
    "reasoning": "Context directly addresses key mechanisms; minor gaps are acceptable."
}
{
    "confidence_score": 0.25,
    "confident": false,
    "reasoning": "Context is tangential; core details are missing."
}

```

A.2 Question Decomposition

Listing 2: Question Decomposition Prompt

You are Knowledge GPT.

STATE: Question Decomposition

Decompose the user's photonics research question into 2-3 highly specific, technically relevant

Output only a valid Python list of strings.

Question: "{question}"

- Use only explicit dimensions: device physics, system impact, implementation challenges.
- Do not add extraneous categories or generalizations.
- Output as a valid Python list of strings.

A.3 Self-Evaluation of Subtopics

Listing 3: Self-Evaluation Prompt

You are Confidence GPT.

STATE: Self-Evaluation

For the subtopic: "{topic}", estimate your confidence in answering using only the given context. Respond with a single float between 0.0 and 1.0 from the set of [0.0, 0.25, 0.5, 0.75, 1.0].

- 1.0 = completely confident
- 0.75 = confident with minor uncertainty
- 0.5 = moderate understanding, needs verification
- 0.25 = partial or limited knowledge
- 0.0 = no meaningful understanding
- If you can answer with high confidence, give a score ≥ 0.75 .
- If any detail is missing, give a low score (< 0.5).

Context:

{base_context}

A.4 Final Answer (Formatting + Safety)

Listing 4: Final Answer Prompt

You are Knowledge GPT.

STATE: Final Answer

Your task is to generate a technically rigorous, well-organized answer to a photonics research question.

- Clearly explain key findings using bullet points and short paragraphs.
- Use only the provided context and your own expertise. Do not hallucinate or fabricate details.

Before answering, make an internal note of the following:

- Retrieval confidence: {confidence:.2f}
- Average similarity of context: {mean_sim:.2f}

Use this format:

- **Key Point 1**
 - Explanation (1)
- **Key Point 2**
 - Explanation (2)

Replace "Key Point" with the actual topic of the point.

Table 7: Finite-State Machine (FSM) states and responsibilities.

State	Purpose / Key Outputs
RELEVANCE	Gatekeeping: determine domain relevance; may terminate early if irrelevant.
CONFIDENCE	Score answerability via strict JSON; also generate a fast-draft snippet for reference.
DECOMPOSITION	Break question into 2–3 concrete subtopics (device physics / system impact / implementation).
SELF_EVAL	Per-subtopic confidence (discrete {0, .25, .5, .75, 1}); decides whether to search online or proceed.
SEARCH_ONLINE	Optional: retrieve 1–k items per low-confidence subtopic; iterate then return to SELF_EVAL.
ANSWER	Compose final, structured answer with citations; optionally prepend a low-confidence disclaimer.
DONE	Terminal state.

A Hyperparameters, Corpus Stats, and Extra Results

A.1 FSM Model Configuration

We used three model roles in the finite-state machine (FSM), configured via `llm_config.py`. Table 8 summarizes the assignments.

The reasoning “effort” knob is only available for `o4-mini` and related models; for `gpt-4o-mini`, default decoding is used. A global temperature override can be applied per run but defaults to the model’s preset `:contentReference[oaicite:0]index=0`.

A.2 Retrieval and Context Parameters

Retrieval was implemented in `context_manager.py` using a dynamic- k strategy. Table 9 lists the key values.

Table 8: FSM model configuration (default “medium” mode).

FSM State	Model	Effort / Temperature
Relevance Check	<code>gpt-4o-mini</code>	default temperature (fast)
Confidence Check	<code>o4-mini</code>	reasoning effort = medium
Knowledge / Answer	<code>o4-mini</code>	reasoning effort = medium
Final Answer	<code>o4-mini</code>	reasoning effort = medium

Table 9: Retrieval hyperparameters.

Parameter	Value
Start k per DB	3
Batch increment	3
Maximum k	12
Similarity threshold	0.75
Top- L context size	12 (unless overridden)
Retry budget	5 FSM iterations (Sec. 4)

Each candidate document chunk is enriched with metadata from `citations.json` and merged into the global evidence pool.

A.3 Prompt and Decoding Policies

Prompts for each state are defined in `conversation_manager.py`. Confidence and self-evaluation prompts return structured JSON with scores in $\{0.0, 0.25, 0.5, 0.75, 1.0\}$, and the FSM transitions accordingly. Role-segmented system prompts enforce rigid output formats (e.g., “Relevant: Yes/No” for relevance):`contentReference[oaicite:2]index=2`.

A.4 Session and Logging Configuration

Sessions are managed under a persistent `sessions/` root (`session_manager.py`) with UUID4 identifiers and JSON logs. Each message entry contains `{role, content, timestamp, usage}`. Fast model: `gpt-4o-mini` is used to generate concise 3–6 word session titles for UI labeling:`contentReference[oaicite:3]index=3`.

A.5 Evaluation Harness

The evaluation script `collect_test_data.py` executes one FSM run per question and logs outputs into a tidy CSV. Key flags:

- `--system-id`: run identifier
- `--reasoning-level`: `{low, medium, high}`
- `--relevance-model`, `--confidence-model`, `--knowledge-model`
- `--temperature`, `--retrieval-k`, `--allow-online-search`
- `--workers`: parallel processes

Each row records: confidence score, citations, fast-draft metrics, FSM outputs, latency, tokens, and cost. Gold source normalization strips PDF extensions and path information for fair matching:`contentReference[oaicite:4]index=4`.

A.6 Corpus Statistics

The ingestion pipeline crawls five tiers of sources (IEEE, Optica/Nature; major conferences; indices; arXiv; Espacenet/USPTO patents). Deduplication uses $\text{SHA-1}(\text{PDF}) \oplus \text{DOI}$ keys. Accessible PDFs are parsed with GROBID; metrics are extracted via deterministic regex and a reasoning pass (`o4-mini`) over 4k-token excerpts:`contentReference[oaicite:5]index=5`.

Together, these hyperparameters and settings fully specify the FSM assistant configuration used in experiments.